

Banco de Dados

COMANDOS DDL

CREATE DATABASE

□ Sintaxe:

```
CREATE DATABASE nome_banco;
```

□ Exemplo:

```
CREATE DATABASE Hospital;
```

```
CREATE DATABASE Farmacia;
```

```
CREATE DATABASE Loja;
```

DROP DATABASE

Exclui uma base de dados;

□ Sintaxe:

```
DROP DATABASE nome_banco;
```

□ Exemplo:

```
DROP DATABASE Hospital;
```

```
DROP DATABASE Farmacia;
```

```
DROP DATABASE Loja;
```

CREATE TABLE

Cria uma tabela no banco;

□ Sintaxe:

```
CREATE TABLE nome_tabela (  
    coluna1          tipo_dado,  
    coluna2          tipo_dado,  
    coluna3          tipo_dado  
);
```

Obs.: Na última coluna não deve haver vírgula.

CREATE TABLE

Cria uma tabela no banco;

▣ Exemplo:

```
CREATE TABLE Pessoa (  
    id            INT,  
    nome          VARCHAR(100),  
    dt_nasc       DATE  
);
```

Tipos de Dados: <https://dev.mysql.com/doc/refman/5.7/en/data-types.html>

TRUNCATE TABLE

Deleta todos os registros da tabela. Não permite a recuperação dos dados excluídos.

□ Sintaxe:

```
TRUNCATE TABLE nome_tabela;
```

□ Exemplo:

```
TRUNCATE TABLE Funcionario;
```

ALTER TABLE

Utilizado para adicionar, deletar e modificar colunas em uma tabela;

Também pode ser utilizado para adicionar ou remover CONSTRAINTS da tabela.

ALTER TABLE

Adicionar uma nova coluna.

□ Sintaxe:

```
ALTER TABLE nome_tabela  
ADD nome_coluna tipo_dado;
```

□ Exemplo:

```
ALTER TABLE Peixe  
ADD peso FLOAT;
```


ALTER TABLE

Excluir uma coluna.

□ Sintaxe:

```
ALTER TABLE nome_tabela
```

```
DROP COLUMN nome_coluna;
```

□ Exemplo:

```
ALTER TABLE Peixe
```

```
DROP COLUMN tamanho;
```

ALTER TABLE

Mudar o tipo de dado da coluna.

□ Sintaxe:

```
ALTER TABLE nome_tabela
```

```
MODIFY COLUMN nome_coluna tipo_dado;
```

□ Exemplo:

```
ALTER TABLE Peixe
```

```
MODIFY COLUMN peso DOUBLE;
```

ALTER TABLE

Renomear a coluna e mudar o tipo.

□ Sintaxe:

```
ALTER TABLE nome_tabela
```

```
CHANGE coluna_antiga coluna_nova tipo_dado;
```

□ Exemplo:

```
ALTER TABLE Convenio
```

```
CHANGE dt_nasc dt_nascimento DATE;
```

Modifica o nome da coluna dt_nasc para dt_nascimento.

ALTER TABLE

Renomear a coluna.

□ Sintaxe:

```
ALTER TABLE nome_tabela
```

```
RENAME COLUMN nome_coluna TO novo_nome;
```

□ Exemplo:

```
ALTER TABLE Produto
```

```
RENAME COLUMN id_categoria TO categoria;
```

Modifica o nome da coluna id_categoria para dt_categoria.

ALTER TABLE

Renomear tabela.

□ Sintaxe:

```
ALTER TABLE nome_tabela  
RENAME TO novo_nome_tabela;
```

□ Exemplo:

```
ALTER TABLE Produto  
RENAME TO dt_nasc dt_nascimento DATE;
```

Modifica o nome da coluna dt_nasc para dt_nascimento.

RENAME TABLE

Renomear tabela.

□ Sintaxe:

```
RENAME TABLE nome_tabela TO novo_nome;
```

□ Exemplo:

```
RENAME TABLE produto TO produtos;
```

DESCRIBE TABLE

Descreve a tabela.

□ Sintaxe:

```
DESCRIBE nome_tabela;
```

□ Exemplo:

```
DESCRIBE produtos;
```

Comentários

Utilizados para explicar comandos SQL.

□ Comentar uma linha:

--Texto a ser comentado

□ Comentar várias linhas:

/*Texto multi-linha
a ser comentado.*/*

COMANDOS DDL - CONSTRAINTS

CONSTRAINTS

- ▣ Especificam regras para os dados nas tabelas;
- ▣ Se alguma ação violar uma *constraint*, esta ação é abortada;
- ▣ Podem ser aplicadas às colunas ou tabelas;
- ▣ Podem ser especificadas na criação da tabela (CREATE TABLE) ou posteriormente com o ALTER TABLE;

CONSTRAINTS

□ Sintaxe:

```
CREATE TABLE nome_tabela (  
    coluna1        tipo_dado    constraint,  
    coluna1        tipo_dado    constraint,  
    coluna1        tipo_dado    constraint  
);
```

CONSTRAINTS

□ As mais utilizadas:

- **NOT NULL** – a coluna não pode ter valor nulo;
- **UNIQUE** – o valor da coluna deve ser único;
- **PRIMARY KEY** – identifica cada linha da tabela (já atribui NOT NULL e UNIQUE);
- **FOREIGN KEY** – aponta para outro registro em outra tabela ou na mesma;
- **CHECK** – os valores devem satisfazer a condição especificada;
- **DEFAULT** – atribui um valor padrão quando nenhum valor é especificado;
- **INDEX** – usado para criar ou recuperar dados de uma tabela rapidamente.

NOT NULL

- ▣ A coluna não aceita valores nulos.
- ▣ Exemplo:

```
CREATE TABLE Cliente (  
    id            INT    NOT NULL,  
    nome          VARCHAR(100) NOT NULL,  
    idade         INT  
);
```

UNIQUE

- ▣ O valor da coluna deve ser único;
- ▣ Tanto UNIQUE quanto PRIMARY KEY garantem a unicidade da coluna;
- ▣ PRIMARY KEY automaticamente possui UNIQUE;
- ▣ Em uma tabela podem haver várias *constraints* UNIQUE, mas apenas uma PRIMARY KEY.

UNIQUE

Exemplo:

```
CREATE TABLE Cliente (  
    id            INT    NOT NULL  UNIQUE,  
    nome          VARCHAR(100),  
    idade         INT  
);
```

ou...

UNIQUE

```
CREATE TABLE Cliente (  
    id            INT    NOT NULL,  
    nome          VARCHAR(100),  
    idade         INT,  
    UNIQUE (id)  
);
```

Ou

```
CREATE TABLE Cliente (  
    id            INT    NOT NULL,  
    nome          VARCHAR(100),  
    idade         INT,  
    CONSTRAINT uc_cliente UNIQUE (id, nome)  
);
```


PRIMARY KEY

- Identifica um registro na tabela.
- Exemplo:

```
CREATE TABLE Cliente (  
    id            INT    PRIMARY KEY,  
    nome          VARCHAR(100),  
    idade         INT  
);
```

ou...

PRIMARY KEY

```
CREATE TABLE Cliente (  
    id            INT,  
    nome          VARCHAR(100),  
    idade         INT,  
    PRIMARY KEY (id)  
);
```

Ou

```
CREATE TABLE Cliente (  
    id            INT,  
    nome          VARCHAR(100),  
    idade         INT,  
    CONSTRAINT pk_cliente PRIMARY KEY (id, nome)  
);
```

Obs.: só deve haver uma chave primária por tabela, a qual pode ser formada por mais de uma coluna.

FOREIGN KEY

□ Um campo (ou conjunto de campos) que se refere à chave primária de outra tabela.

□ Exemplo:

```
CREATE TABLE Pedidos (  
    id            INT    NOT NULL,  
    numero        INT    NOT NULL,  
    id_cliente    INT,  
    PRIMARY KEY (id),  
    FOREIGN KEY (id_cliente) REFERENCES Cliente(id)  
);
```

FOREIGN KEY

- A CONSTRAINT pode ser definida explicitamente.

- Exemplo:

```
CREATE TABLE Pedidos (  
    id            INT    NOT NULL,  
    numero        INT    NOT NULL,  
    id_cliente    INT,  
    PRIMARY KEY (id),  
    CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente)  
    REFERENCES Cliente(id)  
);
```

FOREIGN KEY

▣ Adicionando a chave estrangeira através do ALTER TABLE.

▣ Exemplo:

```
ALTER TABLE Pedidos  
ADD CONSTRAINT fk_pedidos_cliente  
FOREIGN KEY (id_cliente) REFERENCES Clientes (id);
```

fk_pedidos_cliente → nome da CONSTRAINT (você define);

id_cliente → coluna chave estrangeira na tabela;

Clientes → tabela referenciada pela chave estrangeira;

Id → coluna chave primária na tabela referenciada.

FOREIGN KEY

- Excluindo a CONSTRAINT FOREIGN KEY.

- Sintaxe:

```
ALTER TABLE nome_tabela
```

```
DROP FOREIGN KEY nome_constraint;
```

- Exemplo:

```
ALTER TABLE Pedidos
```

```
DROP FOREIGN KEY fk_pedidos_cliente;
```

CHECK

- Limita os valores que podem ser atribuídos a uma coluna.
- Exemplo:

```
CREATE TABLE Persons (  
    id            int            NOT NULL,  
    nome          varchar(255)   NOT NULL,  
    sobrenome     varchar(255),  
    idade         int,  
    CHECK (idade >= 18)  
);
```

Pode ser definido em múltiplas colunas:

```
CONSTRAINT CHK_Person CHECK (age>=18 AND City='Sandnes');
```

CHECK

- ▣ Para ser definida em uma tabela já criada:

```
ALTER TABLE Persons  
ADD CHECK (Age>=18);
```

- ▣ Para deletar a Constraint:

```
ALTER TABLE Persons  
DROP CHECK CHK_PersonAge;
```


DEFAULT

- Define um valor padrão para a coluna.

- Exemplo:

```
CREATE TABLE Persons (  
    id            int            NOT NULL,  
    nome          varchar(255)   NOT NULL,  
    sobrenome     varchar(255),  
    idade         int,  
    cidade        varchar(255)   DEFAULT 'Palmas'  
);
```

DEFAULT

- Podem ser utilizadas funções.

- Exemplo:

```
CREATE TABLE Orders (  
    ID int NOT NULL,  
    OrderNumber int NOT NULL,  
    OrderDate date DEFAULT CURRENT_DATE()  
);
```

INDEX

- Cria índices na tabela.
- Permitem que as consultas sejam realizadas mais rapidamente.
- Sintaxe:

```
CREATE (UNIQUE) INDEX index_name  
ON table_name (column1, column2, ...);
```

- Exemplo:

```
CREATE INDEX idx_nome  
ON Pessoa (nome);
```

Banco de Dados

COMANDOS DML

SELECT

Seleciona dados do banco de dados;

□ Sintaxe:

```
SELECT coluna1, coluna2, ...
```

```
FROM nome_tabela;
```

□ Exemplo:

```
SELECT nome, idade
```

```
FROM clientes;
```

SELECT DISTINCT

Retorna apenas valores diferentes;

□ Sintaxe:

```
SELECT DISTINCT coluna1, coluna2, ...  
FROM nome_tabela;
```

□ Exemplo:

```
SELECT DISTINCT cidade FROM clientes;  
SELECT COUNT(DISTINCT Country) FROM Customers;
```

WHERE

Cláusula para filtrar registros;

□ Sintaxe:

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE condition;
```

□ Exemplo:

```
SELECT * FROM Customers
```

```
WHERE Country = 'Mexico';
```

WHERE (Operadores)

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

AND, OR e NOT

O WHERE pode ser combinado com esses operadores;

□ Sintaxe AND:

SELECT *column1, column2, ...*

FROM *table_name*

WHERE *condition1 AND condition2 AND condition3 ...;*

□ Sintaxe OR:

SELECT *column1, column2, ...*

FROM *table_name*

WHERE *condition1 OR condition2 OR condition3 ...;*

AND, OR e NOT

O WHERE pode ser combinado com esses operadores;

□ Sintaxe NOT:

SELECT *column1, column2, ...*

FROM *table_name*

WHERE NOT *condition*;

ORDER BY

Ordena o resultado;

□ Sintaxe:

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
ORDER BY column1, column2, ... ASC|DESC;
```

□ Exemplo:

```
SELECT * FROM Customers
```

```
ORDER BY Country DESC;
```

```
SELECT * FROM Customers
```

```
ORDER BY Country ASC, CustomerName DESC;
```

INSERT INTO

Inserir registros na tabela;

□ Sintaxe:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

□ Exemplo:

```
INSERT INTO Customers (CustomerName, ContactName,
Address, City, PostalCode, Country)
VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen
21', 'Stavanger', '4006', 'Norway');
```

NULL

- Valor de um campo sem valor;
- É diferente de zero e campos com espaço;
- Não é possível fazer comparações com =, <, <>, etc;
- Para fazer comparações utiliza-se IS NULL ou IS NOT NULL.

IS NULL

Verifica se um campo é nulo;

□ Exemplo:

```
SELECT CustomerName, ContactName, Address  
FROM Customers  
WHERE Address IS NULL;
```

IS NOT NULL

Verifica se um campo **não** é nulo;

□ Exemplo:

```
SELECT CustomerName, ContactName, Address  
FROM Customers  
WHERE Address IS NOT NULL;
```

UPDATE

Modifica um registro na tabela;

□ Sintaxe:

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

□ Exemplo:

```
UPDATE Customers  
SET ContactName = 'Alfred Schmidt', City = 'Frankfurt'  
WHERE CustomerID = 1;
```


UPDATE

É possível atualizar múltiplos registros;

□ Exemplo:

```
UPDATE Customers  
SET PostalCode = 00000  
WHERE Country = 'Mexico';
```

Obs.: Se a cláusula **WHERE** não for especificada, todos os registros serão atualizados.

DELETE

Deleta um registro na tabela;

□ Sintaxe:

DELETE FROM *table_name* WHERE *condition*;

□ Exemplo:

DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';

DELETE

O comando abaixo apaga todos os registros;

□ Exemplo:

```
DELETE FROM Customers;
```

LIMIT

Especifica quantos registros serão retornados;

□ Sintaxe:

```
SELECT column_name(s)  
FROM table_name  
WHERE condition  
LIMIT number;
```

□ Exemplo:

```
SELECT * FROM Customers  
WHERE Country='Germany'  
LIMIT 3;
```

SQL – FUNÇÕES DE AGREGAÇÃO

MIN()

Função que retorna o menor valor da coluna selecionada;

□ Sintaxe:

```
SELECT MIN(column_name)  
FROM table_name  
WHERE condition;
```

□ Exemplo:

```
SELECT MIN(Price) AS SmallestPrice  
FROM Products;
```

MAX()

Função que retorna o maior valor da coluna selecionada;

□ Sintaxe:

```
SELECT MAX(column_name)  
FROM table_name  
WHERE condition;
```

□ Exemplo:

```
SELECT MAX(Price) AS LargestPrice  
FROM Products;
```

COUNT()

Função que retorna o número de linhas que correspondem com o critério especificado;

□ Sintaxe:

```
SELECT COUNT(column_name)  
FROM table_name  
WHERE condition;
```

□ Exemplo:

```
SELECT COUNT(ProductID)  
FROM Products;
```


AVG()

Função que retorna o valor médio de uma coluna numérica;

□ Sintaxe:

```
SELECT AVG(column_name)  
FROM table_name  
WHERE condition;
```

□ Exemplo:

```
SELECT AVG(Price)  
FROM Products;
```

SUM()

Função que retorna a soma total de uma coluna numérica;

□ Sintaxe:

```
SELECT SUM(column_name)  
FROM table_name  
WHERE condition;
```

□ Exemplo:

```
SELECT SUM(Quantity)  
FROM OrderDetails;
```

Banco de Dados

SQL – JOINS

JOIN

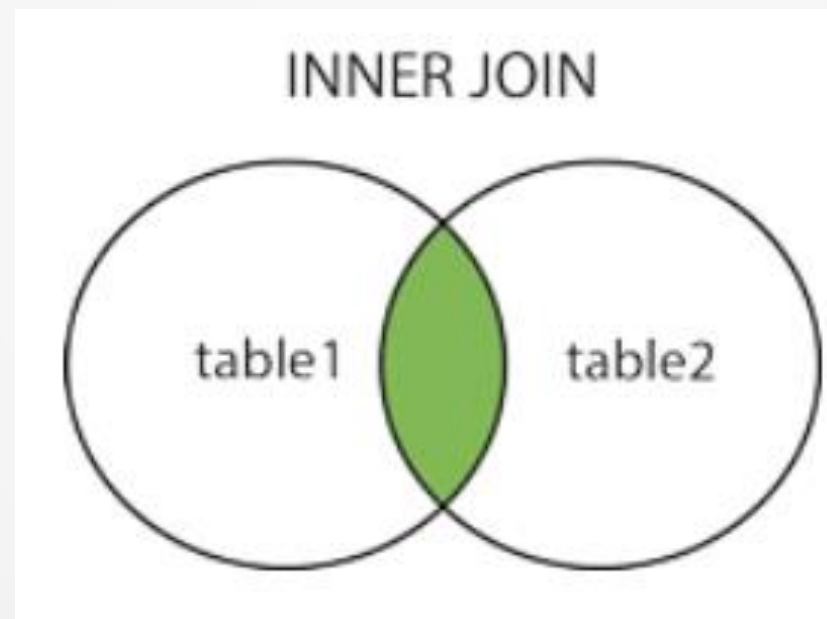
Combina linhas de duas ou mais tabelas, baseado em uma coluna;

□ Exemplo:

```
SELECT Orders.OrderID, Customers.CustomerName,  
Orders.OrderDate  
FROM Orders  
INNER JOIN Customers ON Orders.CustomerID=Customers.CustomerID;
```

INNER JOIN

Retorna os registros cujos valores coincidem em ambas as tabelas.



INNER JOIN

□ Sintaxe:

```
SELECT column_name(s)  
FROM table1  
INNER JOIN table2  
ON table1.column_name = table2.column_name;
```

□ Exemplo:

```
SELECT Orders.OrderID, Customers.CustomerName  
FROM Orders  
INNER JOIN Customers ON Orders.CustomerID =  
Customers.CustomerID;
```

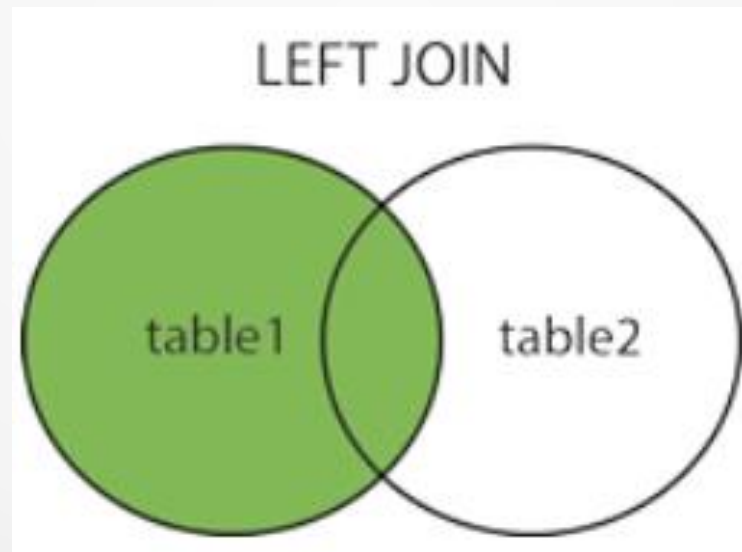
JOIN COM 3 TABELAS

Exemplo:

```
SELECT Orders.OrderID, Customers.CustomerName,  
Shippers.ShipperName  
FROM ((Orders  
INNER JOIN Customers ON Orders.CustomerID =  
Customers.CustomerID)  
INNER JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID);
```

LEFT JOIN

Retorna todos os registros da tabela à esquerda, e os registros que coincidem com a tabela à direita.



LEFT JOIN

□ Sintaxe:

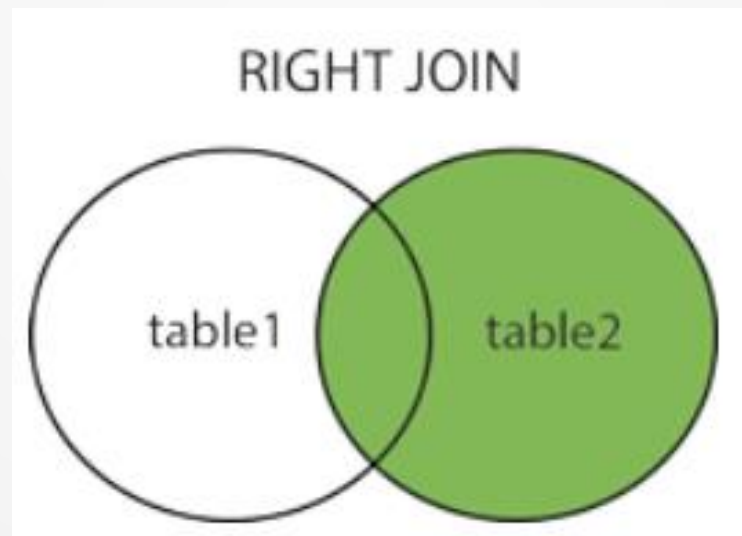
```
SELECT column_name(s)  
FROM table1  
LEFT JOIN table2  
ON table1.column_name = table2.column_name;
```

□ Exemplo:

```
SELECT Customers.CustomerName, Orders.OrderID  
FROM Customers  
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID  
ORDER BY Customers.CustomerName;
```

RIGHT JOIN

Retorna todos os registros da tabela à direita, e os registros que coincidem com a tabela à esquerda.



RIGHT JOIN

□ Sintaxe:

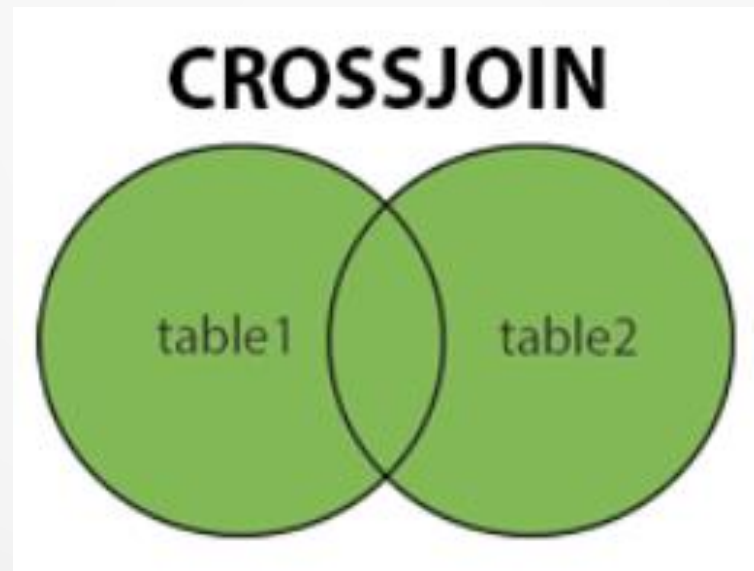
```
SELECT column_name(s)  
FROM table1  
RIGHT JOIN table2  
ON table1.column_name = table2.column_name;
```

□ Exemplo:

```
SELECT Customers.CustomerName, Orders.OrderID  
FROM Customers  
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID  
ORDER BY Customers.CustomerName;
```

CROSS JOIN

Retorna todos os registros de ambas as tabelas.



CROSS JOIN

□ Sintaxe:

```
SELECT column_name(s)  
FROM table1  
CROSS JOIN table2;
```

□ Exemplo:

```
SELECT Customers.CustomerName, Orders.OrderID  
FROM Customers  
CROSS JOIN Orders;
```

Banco de Dados

SQL – UNION, GROUP BY, HAVING

UNION

- Combina o resultado de duas ou mais declarações SELECT.
- Cada SELECT deve ter o mesmo número de colunas;
- As colunas devem ter tipos de dados similares;
- As colunas nos dois SELECTs devem estar na mesma ordem.

UNION

□ Sintaxe:

```
SELECT column_name(s) FROM table1  
UNION  
SELECT column_name(s) FROM table2;
```

□ Exemplo:

```
SELECT City FROM Customers  
UNION  
SELECT City FROM Suppliers  
ORDER BY City;
```


UNION ALL

- Retorna valores duplicados.

- Sintaxe:

```
SELECT column_name(s) FROM table1  
UNION ALL  
SELECT column_name(s) FROM table2;
```

- Exemplo:

```
SELECT City FROM Customers  
UNION ALL  
SELECT City FROM Suppliers  
ORDER BY City;
```

UNION com WHERE

□ O exemplo abaixo retorna todas as cidades Alemãs.

□ Exemplo:

```
SELECT City, Country FROM Customers  
WHERE Country='Germany'  
UNION  
SELECT City, Country FROM Suppliers  
WHERE Country='Germany'  
ORDER BY City;
```

GROUP BY

- Permite agrupar linhas que têm o mesmo valor em determinada coluna;
- Permite identificar o número de clientes de cada país, por exemplo;
- Frequentemente utilizado com funções de agregação (COUNT(), MAX(), MIN(), etc.).

GROUP BY

□ Sintaxe:

```
SELECT column_name(s)  
FROM table_name  
WHERE condition  
GROUP BY column_name(s)  
ORDER BY column_name(s);
```

□ Exemplo:

```
SELECT COUNT(id_cliente), estado  
FROM Clientes  
GROUP BY estado;
```

HAVING

□ Foi criado porque o WHERE não pode ser utilizado com funções de agregação.

□ Sintaxe:

```
SELECT column_name(s)  
FROM table_name  
WHERE condition  
GROUP BY column_name(s)  
HAVING condition  
ORDER BY column_name(s);
```

HAVING

Exemplo:

```
SELECT COUNT(id_cliente), estado  
FROM Clientes  
GROUP BY estado  
HAVING COUNT(id_cliente) > 5;
```

Obs.: lista o número de clientes por estado.